

AI 504 – A decider for truth in a model for $\mathcal{A}$

Stefano Volpe

2026-03-18

We work with $\mathcal{A}$, the language of “All p are q ”. Our goal is to use Haskell to implement a decider for $\models$, the “truth in a model” relation between models and individual formulas.

Grading. The submitted code will be inspected and tested. A grade of 4 or higher means that you have submitted a compiling program declaring all identifiers you are asked to implement. On top of which, a 7 means that you have understood all questions. A 10 means a correct answer to at least two problems, and a twelve means a correct answer to all problems. Grades of 0 and 2 are awarded on an ad-hoc basis. The total problem set grade will be an average of the programming grade and the written homework grade.

Instructions. All instructions from the parent problem set apply. Any way of receiving “intelligent” help via a computer is expressly forbidden. The following is the only allowed import statement: you can use any declaration exported by this module.

```
1 import Data.Set
```

As you might be able to observe by hovering on the `Set` module name in your code editor, this module is part of the `containers` package. So, when using `Hoogle`, you can use that as a filter in the dropdown menu.

Problem 1: The type of sentences

Define a unary type constructor `Sentence` following Definition 1.1. If p is a type, then `Sentence p` must be a type representing all possible sentences of logic $\mathcal{A}$ using p as a signature. In particular, if $a :: p$ and $b :: p$, then there should be some expression `AllAre a b :: p`. Only expressions constructed using such a rule should have type `Sentence p`.¹

Example 1.1: Let `Char` be the set of all Unicode code points. Then:

$$\varphi := \text{All 'p' are 'q'}$$

is a sentence over signature `Char`.

```
1 phi :: Sentence Char
2 phi = AllAre 'p' 'q'
```

Problem 2: The type of models

Define a binary type constructor `Model` following Definition 1.3. If p and m are types, then `Model p m` must be a type representing all possible models for logic $\mathcal{A}$ with signature p and universe m . Let f be a function whose domain is p and whose codomain is the type of all finite² sets of terms in m , then there should be some expression `Mod f :: Model p m`. Only expressions constructed using such a rule should have type `Model p m`. Search the imported library for the “type of finite sets of expressions of a certain type”.

Note that terms of `Model p m` only need to specify the interpretation function $\llbracket \cdot \rrbracket$ (named f above) from the pair $\mathcal{M} = (M, \llbracket \cdot \rrbracket)$ as described in Definition 1.3. This is because the universe M is already present in the type as `Model p m as m`.

¹Haskell has a few ways of cheating the type system. We will always ignore those during the course.

²We stick to finite sets, despite the power set used in Definition 1.3 containing infinite sets, as well. This is because some algorithmic operations are easier with finite sets.

Example 2.1: Let Bool and String be the set of all boolean values and countable Unicode strings respectively. We use Bool as signature. Then:

$$\mathcal{M}_0 := (\text{String}, \llbracket _ \rrbracket_0 : \text{Bool} \rightarrow \mathcal{P}(\text{String}))$$

$$\forall b \in \text{Bool}. \llbracket b \rrbracket_0 := \emptyset$$

$$\mathcal{M}_1 := (\mathbb{Z}, \llbracket _ \rrbracket_1 : \mathbb{Z} \rightarrow \mathcal{P}(\mathbb{Z}))$$

$$\forall z \in \mathbb{Z}. \llbracket z \rrbracket_1 := \{-|z|, \dots, |z|\}$$

are models.

```
1 m0 :: Model Bool String
2 m0 = Mod (const empty)
3
4 m1 :: Model Integer Integer
5 m1 = Mod (\ z -> fromList
6           [- abs z .. abs z])
```

Problem 3: A decider for $\models$

Let U be some set with a subset $Y \subseteq U$. We can call (U, Y) a *decision problem*, and every element of U an *instance* of decision problem U . The elements in Y are *yes-instances*, i.e. instances for which the correct answer is “Yes!”. The elements in $U \setminus Y$ are *no-instances*, i.e. instances for which the correct answer is “No!”. A decider for (U, Y) is a computer program which accepts elements of U as inputs. It must terminate and return True when the input is in Y , while it must terminate and return False when the input is not in Y .

Fix a signature P and a universe M . Consider the decision problem (U, Y) where:

$$U := \{(\mathcal{M}, \varphi) \mid \mathcal{M} \text{ is a model and } \varphi \text{ is a formula}\}$$

$$Y := \{(\mathcal{M}, \varphi) \in U \mid \mathcal{M} \models \varphi\}$$

Define an infix binary operator $|=$ which works as a decider for this decision problem. This operator is polymorphic in a generic type p representing P , and in a type m implementing the `Ord` type class³ and representing M . The operator should be curried: the two expressions representing $\mathcal{M}$ and φ must be passed as the left and right arguments respectively.

Example 3.1: Consider the previous example again, and remember that the Haskell Language Server can evaluate snippets prefixed with `>>>` in comments (unlike in Python, no need to use an interpreter!).

$$\mathcal{M}_0 \models \text{All True are False} \stackrel{\text{def}}{=} \llbracket \text{True} \rrbracket_0 \subseteq \llbracket \text{False} \rrbracket_0$$

$$\stackrel{\text{def}}{=} \emptyset \subseteq \emptyset$$

$$\mathcal{M}_1 \models \text{All } -3 \text{ are } 2 \stackrel{\text{def}}{=} \llbracket -3 \rrbracket_1 \subseteq \llbracket 2 \rrbracket_1$$

$$\stackrel{\text{def}}{=} \{-3, \dots, 3\} \subseteq \{-2, \dots, 2\}$$

```
1 -- >>> m0 |= AllAre True False
2 -- True
3 -- >>> m1 |= AllAre (-3) 2
4 -- False
```

The first statement holds, but the second does not.

Note that implementing a decider means implementing a function that returns a truth value. This is very different from an implementation of the truth predicate in Definition 1.4, which would return an object representing the mathematical statement $\mathcal{M} \models \varphi$, rather than its mere truth value.

³We stick to universes which are *totally ordered*, despite Definition 1.3 allowing for universes which are not totally ordered, as well. This is because some algorithmic operations on the Haskell type for finite sets we are using make this assumption. You do not need to worry about what a total order is, or how the `Ord` type class is defined.